

EXPERIMENT: Minimax algorithm for gaming

Aim

To implement the Minimax algorithm for gaming using Python and develop an intelligent computer player for Tic-Tac-Toe.

Objectives

1. To understand the concept of the Minimax algorithm.
2. To represent game states using a game tree.
3. To evaluate possible future moves.
4. To select the optimal move for the computer.
5. To understand MAX and MIN decision-making in AI games.

Algorithm

1. Initialize the Tic-Tac-Toe board.
2. Define all possible winning combinations.
3. Check whether the current game state is a win, loss, or draw.
4. Assign scores:
 - Computer win = +1
 - Human win = -1
 - Draw = 0
5. Generate all possible moves.
6. For the computer, use the MAX operation to select the highest score.
7. For the human player, use the MIN operation to select the lowest score.
8. Recursively evaluate all possible game states.
9. Select the move with the best score.
10. Continue until the game reaches a terminal state.

CODE

```
board = [' ' for _ in range(9)]

def display():
    print()
    print(board[0], '|', board[1], '|', board[2])
    print('--+---+--')
    print(board[3], '|', board[4], '|', board[5])
```

```

print('--+---+')
print(board[6], '|', board[7], '|', board[8])
print()
def winner(player):
    wins = [
        (0,1,2), (3,4,5), (6,7,8),
        (0,3,6), (1,4,7), (2,5,8),
        (0,4,8), (2,4,6)
    ]
    for a, b, c in wins:
        if board[a] == board[b] == board[c] == player:
            return True
    return False
def minimax(is_maximizing):
    if winner('O'):
        return 1
    if winner('X'):
        return -1
    if ' ' not in board:
        return 0
    if is_maximizing:
        best_score = -1000
        for i in range(9):
            if board[i] == ' ':
                board[i] = 'O'
                score = minimax(False)
                board[i] = ' '
                best_score = max(best_score, score)
        return best_score
    else:
        best_score = 1000

```

```

    for i in range(9):
        if board[i] == ' ':
            board[i] = 'X'
            score = minimax(True)
            board[i] = ' '
            best_score = min(best_score, score)
    return best_score

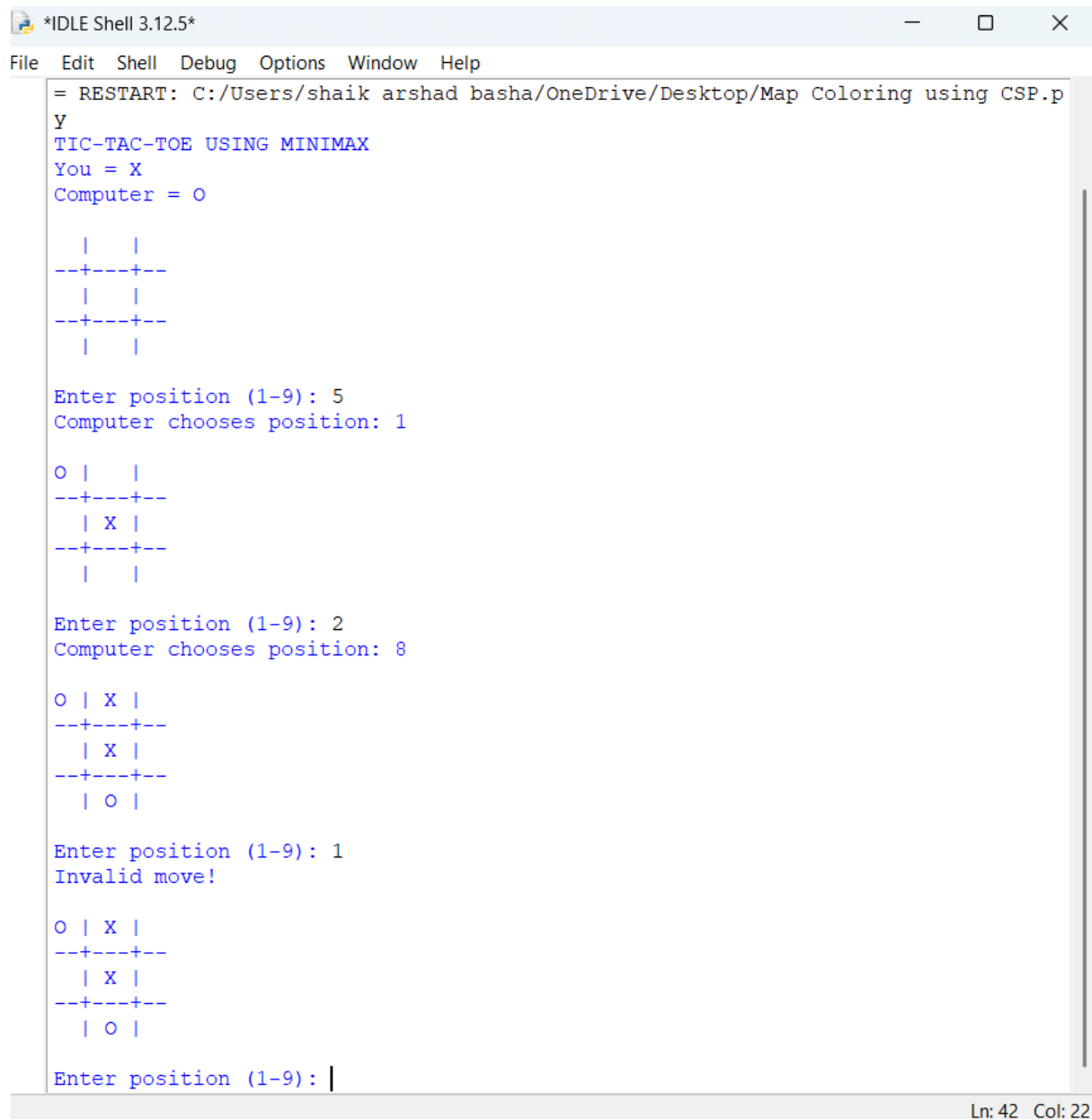
def best_move():
    best_score = -1000
    move = -1
    for i in range(9):
        if board[i] == ' ':
            board[i] = 'O'
            score = minimax(False)
            board[i] = ' '
            if score > best_score:
                best_score = score
                move = i
    return move

print("TIC-TAC-TOE USING MINIMAX")
print("You = X")
print("Computer = O")
while True:
    display()
    try:
        pos = int(input("Enter position (1-9): ")) - 1
    except ValueError:
        print("Invalid input!")
        continue
    if pos < 0 or pos > 8 or board[pos] != ' ':
        print("Invalid move!")

```

```
        continue
    board[pos] = 'X'
    if winner('X'):
        display()
        print("You Win!")
        break
    if ' ' not in board:
        display()
        print("Draw!")
    move = best_move()
    board[move] = 'O'
    print("Computer chooses position:", move + 1)
    if winner('O'):
        display()
        print("Computer Wins!")
        break
    if ' ' not in board:
        display()
        print("Draw!")
        break
```

OUTPUT



```
*IDLE Shell 3.12.5*
File Edit Shell Debug Options Window Help
= RESTART: C:/Users/shaik arshad basha/OneDrive/Desktop/Map Coloring using CSP.p
y
TIC-TAC-TOE USING MINIMAX
You = X
Computer = O

  |  |
--+--+
  |  |
--+--+
  |  |

Enter position (1-9): 5
Computer chooses position: 1

O |  |
--+--+
  | X |
--+--+
  |  |

Enter position (1-9): 2
Computer chooses position: 8

O | X |
--+--+
  | X |
--+--+
  | O |

Enter position (1-9): 1
Invalid move!

O | X |
--+--+
  | X |
--+--+
  | O |

Enter position (1-9): |
```

Ln: 42 Col: 22

Result

The Minimax algorithm was successfully implemented for Tic-Tac-Toe. The computer evaluates possible moves and selects the optimal move based on future game states.

Conclusion

The experiment demonstrates that the Minimax algorithm enables an AI agent to make intelligent decisions in competitive games by considering possible future outcomes. It is useful in game-playing AI and decision-making systems.